Code walkthrough — 15 minutes

Updated 2026-08-31 (Round 2b: floating, fixed-then-floating and stepped bonds; instrument-type dispatch; the delivery-quality pass).

Speaking aid for the session with Mario and the engineering team, 2026-08-25. Supersedes the 2026-08-17 version, which covered only the plain-bond chain. Everything claimed below is backed by the automatic suite: 223 checks, about 19 seconds.

Opening line: "The code is organised so that the mathematics lives in one place and each bond type is a short file that just names its inputs. Everything from the August sample still produces identical numbers — and there are now three more bond types and two ways to call it from outside Python. Here is the tour."

Where each legacy engine went — the reconstruction map

Read this first if the question is "is this the same code that produced last month's numbers?" The answer is yes, and each line is checked by a test rather than by inspection.

Legacy file	New core location	Old path	Asset wrapper	Endpoint type
`pricing/bond_price.py`	`core/pricing/analytical.py` + `cashflows` + `discounting`	shim	`assets/corporate/vanilla.py`	`vanilla`
`pricing/lattice.py`	`core/pricing/tree.py`	shim	`callable` / `puttable` / `sinking`	those three
`pricing/frn.py`	`core/pricing/floating.py`	shim	`assets/corporate/floating.py`	`floating`
`pricing/hybrid.py`	`core/pricing/hybrid.py`	shim	`assets/corporate/hybrid.py`	`fixed_to_floating`
`pricing/coupon_schedule.py`	`core/pricing/coupon_schedule.py`	shim	`assets/corporate/stepped.py`	`stepped`

Four claims, each pinned:

1. the numerical body is **byte-identical** below the docstring (hybrid's two import lines are the one exception, and both targets are asserted to be *the same objects*);
2. every shim re-exports **the same object** — `a is b`, not an equal reimplementation;
3. every endpoint result equals the direct wrapper call with `==`, per instrument type;
4. the production output files are compared **by hash**, before and after every commit.

And the path a bond actually takes, end to end:

```
workbook coupon family      Coupon_Formula2 on the Corporate Bonds tab
  -> router / overrides     dataio/coupon_types.py, dataio/term_overrides.py
  -> core engine            core/pricing/{analytical,tree,floating,hybrid,coupon_schedule}
  -> asset wrapper          assets/corporate/*.py     units + named inputs, no arithmetic
  -> endpoint type          endpoints/  one request in, one response out
  -> driver / JSON output   scripts/*.py
  -> evidence               tests + hashed CSVs + the disposition sidecars
```

What changed numerically, in one place

```
moving the three engines            nothing at all — five driver CSVs byte-identical
the GBP par-yield units correction  two GBP securities, and the headline counts
the callable routing fix            no priced number; one bond went from invisible to named
```

The tour, in order

1. `src/pricer/__init__.py` — the map (30 seconds)

Three layers, and the split is the point:

```
core/       the mathematics, shared by every bond type      (~80% of the code)
assets/     one short file per bond type, no arithmetic     (~20%)
endpoints/  the outside world: one request in, one result out
```

Say: "*If you only remember one thing — a new bond type is a new file in `assets/`, not a new engine.*"

2. `assets/corporate/` — one file per bond type (3 min)

Open `vanilla.py`, then `callable.py`, then `floating.py`. They look the same on purpose: one simple function per output, the same names the legacy Monthly sheet uses. There are now **seven** of these files — vanilla, stepped, floating, hybrid, callable, puttable, sinking.

```
calculated_price   implied_oas   duration   dv01   convexity   widening   tightening
```

Then point at what differs, which is only ever the bond's own terms: `callable.py` takes a **call schedule**, `puttable.py` a **put schedule**, `sinking.py` a **redemption schedule**, `hybrid.py` a **switch date and a post-switch margin**, `stepped.py` a **coupon table**. None of those files does any arithmetic; they name their inputs, convert units and pass the terms down.

`floating.py` is the one worth pausing on, because its input list differs in a way that explains the product: **there is no `coupon`**. A floating coupon is not known in advance, so the fixed rate is replaced by a quoted margin plus (optionally) the one coupon already fixed at the last reset. That single difference is the whole instrument.

Open one function to show the numbered `Inputs` block with units in capitals — this layer speaks the legacy sheet's units (coupon in PERCENT, prices per 100, spreads in BASIS POINTS); the engines underneath work in decimals.

For the engineers: every one of these is a pure function — no state, no globals, one bond per call. A portfolio is embarrassingly parallel with no design change.

3. `core/pricing/tree.py` — one engine, three products (4 min)

This is the heart of the new work. A callable, a puttable and a sinking-fund bond are the same calculation with a different right attached:

```
call right   the issuer caps the value at the call price      min(value, price)
put right    the holder floors it at the put price            max(value, price)
sinking      the issuer retires a fraction f at the price     (1-f)·value + f·min(value, price)
```

Worth saying out loud: **at f = 1 the sinking rule *is* the call rule** — and the code produces the identical value to the last bit, which is one of the tests.

Why one engine and not three: fourteen rows in the workbook carry two rights at once (`CALL/SINK`, `CALL/PUT`). Three separate engines could not price those without copying each other.

If asked how the tree is built: it is calibrated so that it reprices the input curve's own discount factors exactly — arbitrage-free by construction, not by assumption.

4. `core/` — the rest, bottom-up (3 min)

- `utils/dates.py` — the calendar. One year is 364 days, one half-year 182. That is the legacy system's own convention, kept deliberately; changing it would break every validated number.
- `pricing/cashflows.py` — the cash-flow table and **the one** accrued-interest formula that every engine shares.
- `pricing/discounting.py` — the corrected discounting that reprices a curve's own par bonds to exactly 100.
- `pricing/analytical.py` — the plain-bond price: dates → cash flows → discount → sum. About 25 lines of orchestration.
- `risk/sensitivities.py` — duration, DV01, convexity as pure arithmetic on three prices. Point it at *any* pricing function; the tree uses the same one.
- `market/curves.py` — `resolve_curve(currency, date, frequency)`: the one place a bond gets its own-currency curve, and the one place a missing curve is refused rather than substituted.

5. `endpoints/` + `integrations/excel_vba/` — calling it from outside (3 min)

```
Excel cells → VBA bridge → request.json → analyze_vanilla_payload() → response.json → cells
```

`endpoints/main.py` is one function: a dictionary in, a dictionary out, and it never raises — failures come back as a status and a reason. `contracts.py` is the request/response shape, standard library only. `pricing.py` arranges the calls. **No formula anywhere in this layer.**

Two things to demonstrate if there is time: - `integrations/excel_vba/examples/` — a real request and the real response it produced; - `Run-BridgeTests.ps1` — 23 checks driving actual Excel, including a live call into Python.

The line that matters for the cloud move: the spreadsheet knows one setting, a command to run. Point it at a packaged executable or an HTTP service and nothing else changes.

6. Proof, then close (2 min)

- `src/pricing/*.py` — the old module paths are now thin shims. Every existing script, driver and test still works unchanged.
- Migration safety: production output files are compared **by cryptographic hash** before and after every change. Identical, three times over.
- `tests/test_pricer_tree_structure.py` — the interesting ones: the new wrapper equals the production calculation with `==`; a bond with no rights prices exactly as a plain bond; callable ≤ plain ≤ puttable.

Numbers to have ready

Claim	Number
Automatic checks	**300** in ~24 s (223 before this round)
Behaviour change from the restructuring	**zero** — production CSVs hash-identical
Excel bridge	**23/23**, on real Excel, including a live Python round trip
Cross-platform	response file **byte-identical** on Windows and Linux
Volatility, call-active bond	~10 cents of price, or ~1 bp of spread, per vol point
Bond types the interface reaches	**7**, through one entry point and one request format
Bonds added by the GBP units fix	**2** — one blocked, one silently skipped

---

Likely questions — answers ready

- "*Why 364-day years?*" — the legacy engine's own convention. We keep it so every number reconciles. Real day-count labels are carried as data and can drive a future layer.
- "*Why is a callable bond's duration shorter?*" — the issuer's right to repay caps how much the price can rise when rates fall, so the bond moves less.
- "*Can we price a bond that is both callable and sinkable?*" — the engine can hold both sets of terms. Where the two land on the same date we currently refuse, because the order of the two rights changes the answer and no order is tested yet. That is a deliberate boundary, not a gap in the mathematics.
- "*Why refuse instead of assuming?*" — every refusal in the code is a case where a silent assumption would produce a plausible wrong number: a missing curve, a spreadsheet date sent as a number, terms that contradict each other.
- "*How would this scale to a whole portfolio in the cloud?*" — pure functions, one bond per call, deterministic across platforms, and one entry point that any transport can wrap.
- "*What is not built yet?*" — mortgages wait on the data pull; convertibles need an equity model we do not have; and the demonstration spreadsheet still offers plain bonds only, which is a layout decision rather than a technical one.
- "*How do you know nothing is silently missing?*" — because the count is now mechanical. Every bond in the portfolio must leave the calculation with either a number or a named reason, and that is checked as a **set of identifiers**, not as a total. A total balances even when the wrong bond is in the wrong place, which is exactly how two omissions survived. The check runs on every production run and stops it if anything is unaccounted for.

- "*You said the pound curve was broken. What changed?*" — we were reading that file in the wrong units. Most of the market-data files store rates as decimals; the pound file stores percentages, and we multiplied it by a hundred. The bootstrapper was right to refuse the result; we were wrong to read its complaint as a fact about the data. Both pound bonds now price, and one of them had been missing from the output entirely.